

Final Report

Ryan Waterman

Table of Contents

Model Refinement 3

Error Analysis 5

Comparative Strategy 7

Executive Summary 8

Appendix 9

Model Refinement

Based on the results of the Preliminary Results phase, a fully fine-tuned BERT base model was selected for further refinement in the final stage. This model resulted in the best accuracy, recall, precision, and F1-score, as seen in the table below.

Model Type	Accuracy	Recall	Precision	F1-Score
BERT Full Model without Preprocessing	82.96%	79.94%	85.08%	0.82

Table 1: Optimal Model from Preliminary Results

The intent for the model refinement stage was to iteratively train the full BERT model with different hyperparameters, aiming to increase the metrics identified in Table 1 while minimizing training time. The hyperparameters selected for tuning during this phase were batch size, max input length, and layer freezing. Learning rate will not be addressed in this phase because the scheduler is configured to automatically reduce the learning rate as a function of the validation loss. Note that the number of training epochs for all hyperparameter tuning was held constant at 3 to reduce the number of concurrent changing variables. Additionally, all Confusion Matrices and ROC Curve images for each phase of model refinement can be found in the Appendix.

The first hyperparameter explored during the refinement phase was batch size. During the Preliminary Results phase, this parameter defaulted to 8 for all testing. Batch size controls the number of concurrent inputs that the model is trained on in a given batch, which is typically limited by GPU memory as the entire batch must be stored during the training cycle. The hypothesis was that increasing batch size would drive training time down, as the model could train on larger subsets of the data concurrently. The concern is retaining accuracy, which was also tested. It is important to note that this was the first hyperparameter selected for testing due to the anticipated training speed up. In the case that accuracy was not impacted, all further hyperparameter testing could be completed with a faster iteration speed. The results from this phase are in Table 2, below.

Batch Size	Training Time (m:ss)	Accuracy	Recall	Precision	F1-Score
8	8:40	82.96%	79.94%	85.08%	0.82
32	2:39	83.48%	82.79%	83.95%	0.83
64	1:40	83.76%	83.44%	83.97%	0.84

Table 2: Batch Size Optimization

Going forward, a batch size of 64 will be utilized. This batch size offered the best trade-off between training time and accuracy, with a training time reduction of 80.8% and an overall performance improvement against the other configurations.

The next hyperparameter that was tuned was max input length. The baseline model was trained with a max input length of 512. The additional max input length sizes tested were 2048, 1024, 256, and 128. Table 3, below, highlights the model performance metrics, as well as train and inference time, which will also be a consideration for overall performance. The novel dataset that inference was performed on features 14,468 samples, which is an additional factor when considering inference time. Inference time will also be considered during the evaluation of this hyperparameter, as this drives the number of tokens the model must attend back to during inference. In a production application, both inference speed and accuracy are critical.

Max Input Size	Training Time (m:ss)	Inference Time per Sample (ms)	Accuracy	Recall	Precision	F1-Score
2048	3:05	4.46	84.04%	82.98%	84.78%	0.84
1024	3:39	4.42	83.70%	83.52%	83.81%	0.84
512	1:43	4.41	83.90%	83.74%	84.00%	0.84
256	1:38	4.05	83.68%	82.82%	84.58%	0.84
128	1:40	4.25	83.71%	84.23%	83.36%	0.84

Table 3: Max Input Length Optimization

The initial max input size of 512 provided the best balance between performance, training time, and inference time, therefore it will remain as the selected input length going forward.

The final hyperparameter to tune is layer freezing. The intended benefit of this hyperparameter is training time and accuracy, as less of the model will require weight updates and the model should already understand syntactic sentence structure from pretraining. The following configurations of frozen encoder layers will be tested:

1. Layer 11 (the final encoder layer)
2. Layers 9, 10, and 11 (the final three encoder layers)
3. Layers 6, 7, 8, 9, 10, and 11 (the final six encoder layers)

The pooling layer following the final encoder layers, as well as the classifier layer, will always trainable. The final encoder layers were selected for experimentation because the encoder layers at the bottom of the model are responsible for learning syntax and sentence structure, where the top encoder layers learn features like sentiment and complex relationships between words across the sentence. Table 4, below, contains the results from experimentation.

Trainable Layers	Training Time (m:ss)	Accuracy	Recall	Precision	F1-Score
11	0:34	82.06%	82.08%	82.04%	0.82
9, 10, 11	0:41	83.03%	82.39%	83.46%	0.83
6, 7, 8, 9, 10, 11	1:00	83.65%	83.33%	83.87%	0.84

Table 4: Layer Freezing Optimization

Based on Table 4, there is a positive relationship between the number of trainable layers and model performance. None of the partially frozen models reached the accuracy seen when training the full model, however, there is significant training time savings. The largest constraint on performance is the amount of training data, which has been restricted to reduce training time. Given the improvement in training time and relative performance of the model with layers 6-11 trainable, the opportunity to increase the amount of training data arises. With this configuration, a larger portion of the dataset, or even the full dataset, could be trained on.

The final, optimized model configuration for this application is as follows: batch size 64, max input length 512, and leaving only encoder layers 6-11 and pooling/classification layers trainable. This configuration offers the best trade-off between speed, scalability, and accuracy, assuming the final production model will be trained on the entire 1.6 million sample dataset.

Error Analysis

Generally, the selected model performed well when predicting sentiment based on the Confusion Matrix and ROC curve, below.

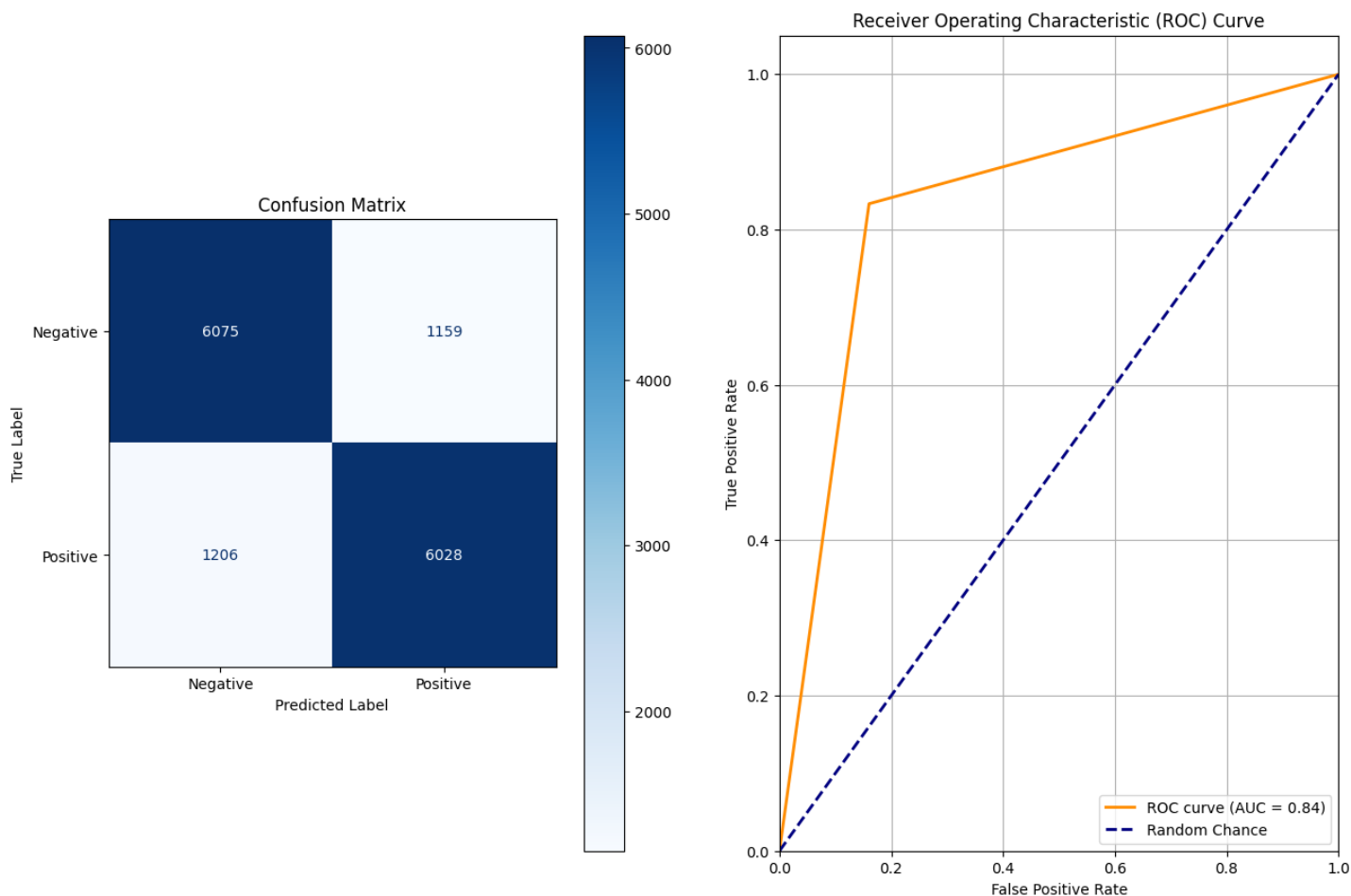


Image 1: CM and ROC Curve for Selected Model

The model predicted a class balance of 49.68% positive and 50.32% negative, which is within 0.4% of the true class balance of 50/50. This is a good indicator that the model has correctly learned the features that contribute to positive or negative sentiment classification but falls short in more nuanced cases that additional training data may bring to light. Table 5, below, details specific samples that the model falsely predicted as positive or negative.

Example	False Positive Examples	False Negative Examples
1	'working inside all day on this sunny hot beautiful day '	"So it turns out my bank card doesn't work here. This is working out to be a very cheap holiday faith no more in two hours! Woot!"
2	"Better start doing some RE revision now. Goodbye social life, I'll be back in an hour "	'Enjoying the heck out of spending time with Amanda. Do i really have to go back tomorrow?"
3	'@jonaskevin haha i understand , oh gosh,, where is Joe?, hope u reply me tomorrow '	"@ComedyQueen had a lot of fun with my friends getting lost on our way to my friend's dad's wake. "

4	" didn't get an award but i did get recognized by all the seniors. They said that they are going to watch the plays because of me "	"when I say 'far away', I meant I threw away my copy "
5	"@yoitsandrea There's a difference between making fun of you and thinking that the picture was funny though. Sorry. "	'@beangirl1965 - So sorry to hear that sweetie! Let me know if you need anything '

Table 5: Examples of False Positives and False Negatives

The examples above indicate two common failure modes for the model; 1. inability to distinguish sarcasm and 2. classifying sentiment based on a single positive or negative sentiment word instead of the general theme. These failure modes highlight the complexity of language, especially that of Twitter data, which is far less structured than novels or new articles. For instance, false negative example 5, when read at face value, contains the negative sentiment word “sorry” with a locally negative sentiment in the phrase “So sorry to hear that...” but when evaluated further, it becomes clear that the general intent of the statement is to be supportive, which is of positive sentiment. False positive example 1 is a perfect example of sarcasm. The statement “working inside all day on this sunny hot beautiful day” is objectively positive, but with the aid of human intuition, it is obvious that being inside on a beautiful sunny day is, in fact, negative. There are also statements like false negative example 4 that, even from the perspective of a human, do not have a clear sentiment. This would be an example of noise in the dataset, as it is unclear from the amount of context provided if this is a positive or negative statement.

Comparative Strategy

BetterHelp's business objective is to utilize Twitter data to generate a targeted ad campaign that aims to integrate ads for their platform into the Twitter feed of users posting overwhelmingly negative tweets. To effectively automate this process with the use of a Natural Language Processing model, there are several critical model requirements. First, model accuracy, which will be the foundation for correctly identifying users to target with therapy ads. In order to reduce wasted capital expenditure on ads for low conversion rate users, the model must reliably and accurately identify negative sentiment tweets. The next requirement is training time. Twitter (currently X) has millions of daily active users, generating terabytes of data. To train a model to be effective at this scale, training time must be low. The final requirement is low inference time, which is critical when predicting the sentiment of thousands of tweets simultaneously.

The baseline model, an SVM classifier, is incredibly quick to train, fitting to the data in just 72 milliseconds. This is about 833 times as fast as the optimized BERT model training time. Similarly, the inference speed on the SVM model is far superior to the optimized BERT model, completing inference on 15000 samples in just 1.1 milliseconds, which averages to 73.3 nanoseconds per sample. This is nearly 58,000 times faster than the BERT model. That said, the dramatic difference in training and inference speed come at the cost of accuracy, which is one of the critical requirements for the application. See Table 6, below, for the overall comparison between the baseline SVM model and the optimized BERT model.

Model Type	Training Time (s)	Inference Time per Sample (s)	Accuracy	Recall	Precision	F1-Score
SVM	0.072	$7.33 * 10^{-8}$	74.05%	74.92%	73.89%	0.74
Optimized BERT	60	0.00441	83.65%	83.33%	83.87%	0.84

Table 6: Baseline Versus Optimized BERT

The training and testing data for SVM and BERT model architectures aid in a data-backed business decision. Although training time, inference time, and accuracy were all presented as critical in the prior section, the cost associated with training time and inference time is different than that of accuracy, recall, precision, or F1-score. With the technology currently available, compute is a resource that has become incredibly cheap. The cost associated with renting the cloud compute to reduce both training time and inference time is trending toward negligible. Additionally, training cost is a one-time fee, and inference, as proven above, is a far less computationally expensive operation. The cost of accuracy disparities between the SVM and BERT models, however, are significant.

Assume 100,000 tweets (50/50 split on sentiment class) were fed into both SVM and BERT models for inference at their current performance levels. The SVM model would incorrectly classify 12,975 negative sentiment tweets, while the BERT model would incorrectly classify 8,175 negative sentiment tweets. These numbers must also be doubled, as BetterHelp is exclusively interested in true negative tweets, therefore false negatives will also result in targeting the wrong audience with ads. This results in 25,950 incorrect classifications for the SVM model and 16,350 incorrect classifications for the BERT model. Further, assume a cost per ad view of \$0.25; the resulting cost of erroneous classifications by the SVM model for 100,000 tweets would be \$6,487.50, while the cost of the BERT model would be \$4087.50. Although there is only a difference of 9.6% in the accuracy of the two models, the resulting wasted cost difference between the two is 37%.

Extrapolate this cost over millions of Tweets, and it becomes very clear that, although BERT is slower and more computationally demanding, the true cost of the model is significantly less than SVM.

For the benefit of cost justification, additional analysis can be performed to determine the potential business benefit utilizing this approach. Continuing under the assumption of 100,000 tweets with a 50/50 sentiment split, the SVM model would correctly identify 37,025 true negative tweets, while BERT would correctly identify 41,825 negative tweets. Assuming a conservative conversion rate of 3%, this will result in 1,111 and 1,255 new customers per 100,000 analyzed tweets for SVM and BERT, respectively. Assuming a cost per week of \$100 for the BetterHelp service, SVM would generate a monthly recurring revenue of \$444,400, while BERT would generate \$502,000. Assuming a \$0.50 cost per ad click and subtracting the previously calculated ad waste and cost per view of true negative prediction ads, the final estimated net monthly income from SVM is \$428,101.25, while BERT is \$486,828.75. This is a difference of \$58,727.50, or about 12.1%, per 100,000 analyzed tweets between the BERT and SVM models.

Executive Summary

In conclusion, the suggested model for BetterHelp's application is the fine-tuned BERT model. This model has an extrapolated, estimated monthly revenue gain of \$486,828.75 per month per 100,000 analyzed tweets. The implementation cost of this model may be marginally higher due to the necessity of rented cloud compute for training, but this is an initial one-time cost. By comparison, the less computationally expensive, faster SVM model has a 37% higher associated cost due to misclassification and results in 12.1% lower estimated monthly revenue. The computational benefits of this model do not warrant the cost associated with the 9.6% accuracy trade-off. Additionally, the BERT model was trained on a small subset of the available data, and the resulting accuracy of the architecture could increase by several percent if the entire training dataset was utilized for the production model.

Appendix

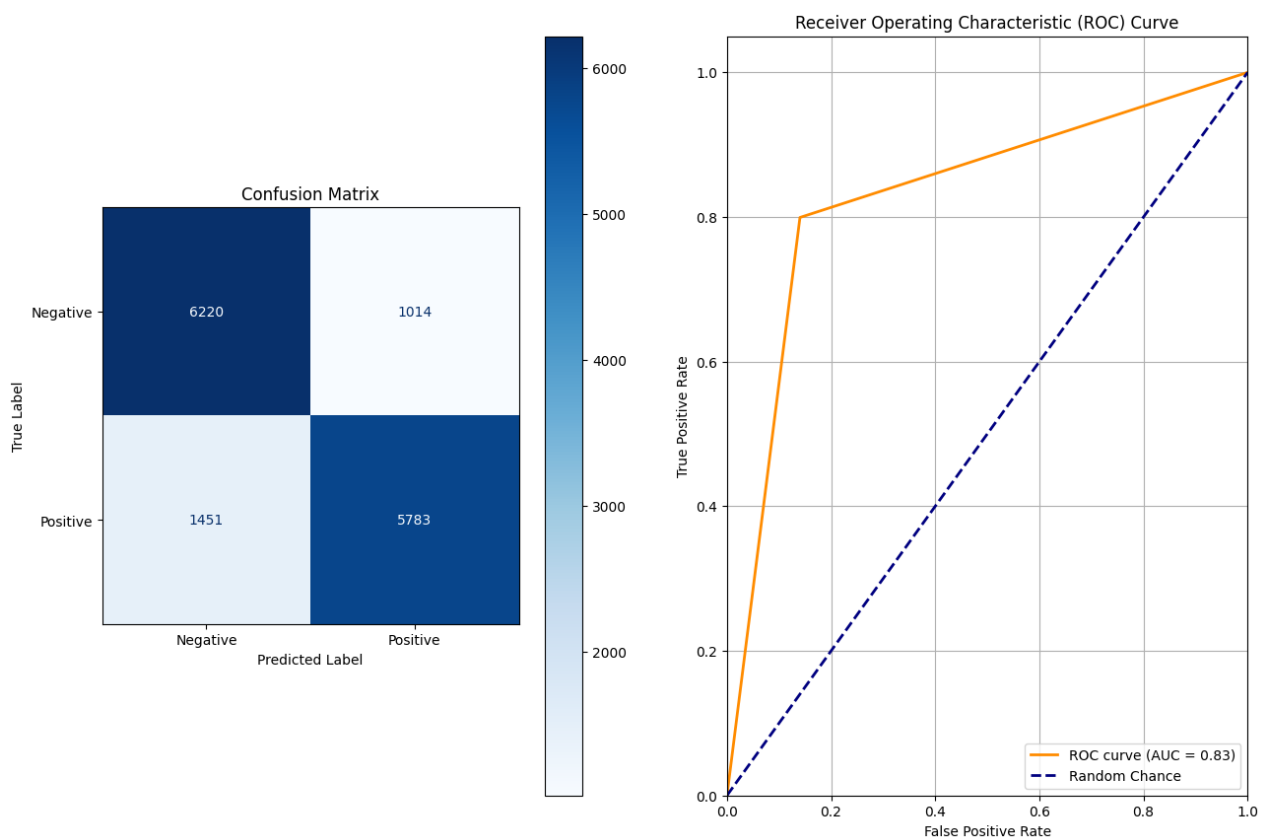


Image 2: CM and ROC Curve for Preliminary Results Optimal Model

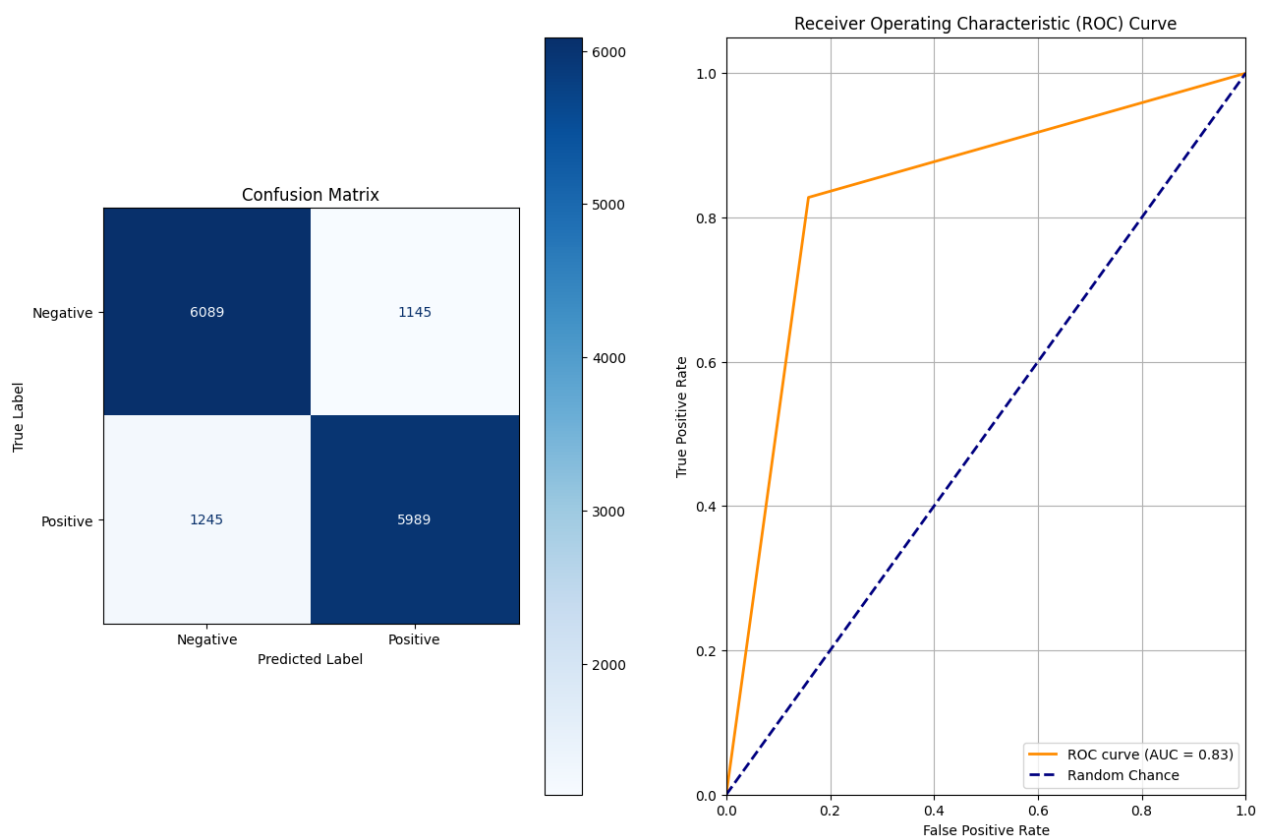


Image 3 CM and ROC Curve for Batch Size Testing (32)

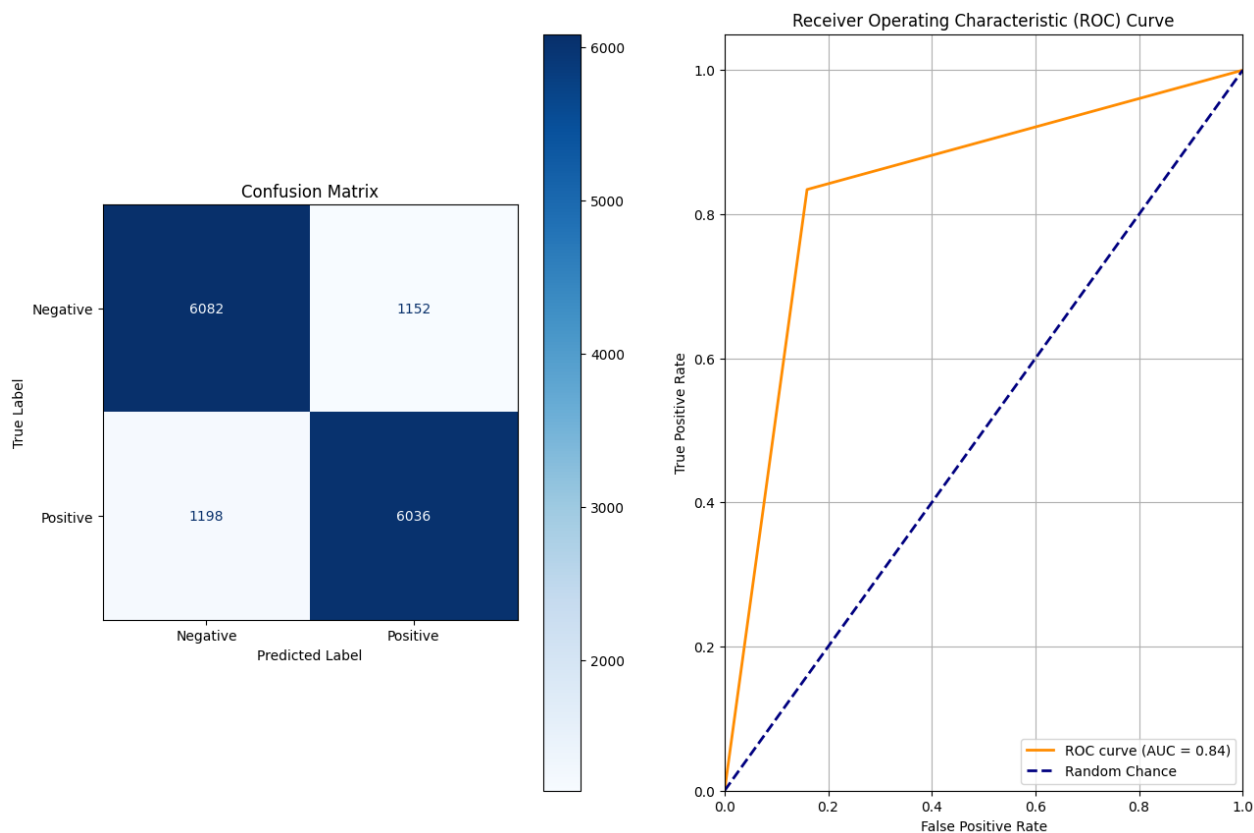


Image 4: CM and ROC Curve for Batch Size Testing (64)

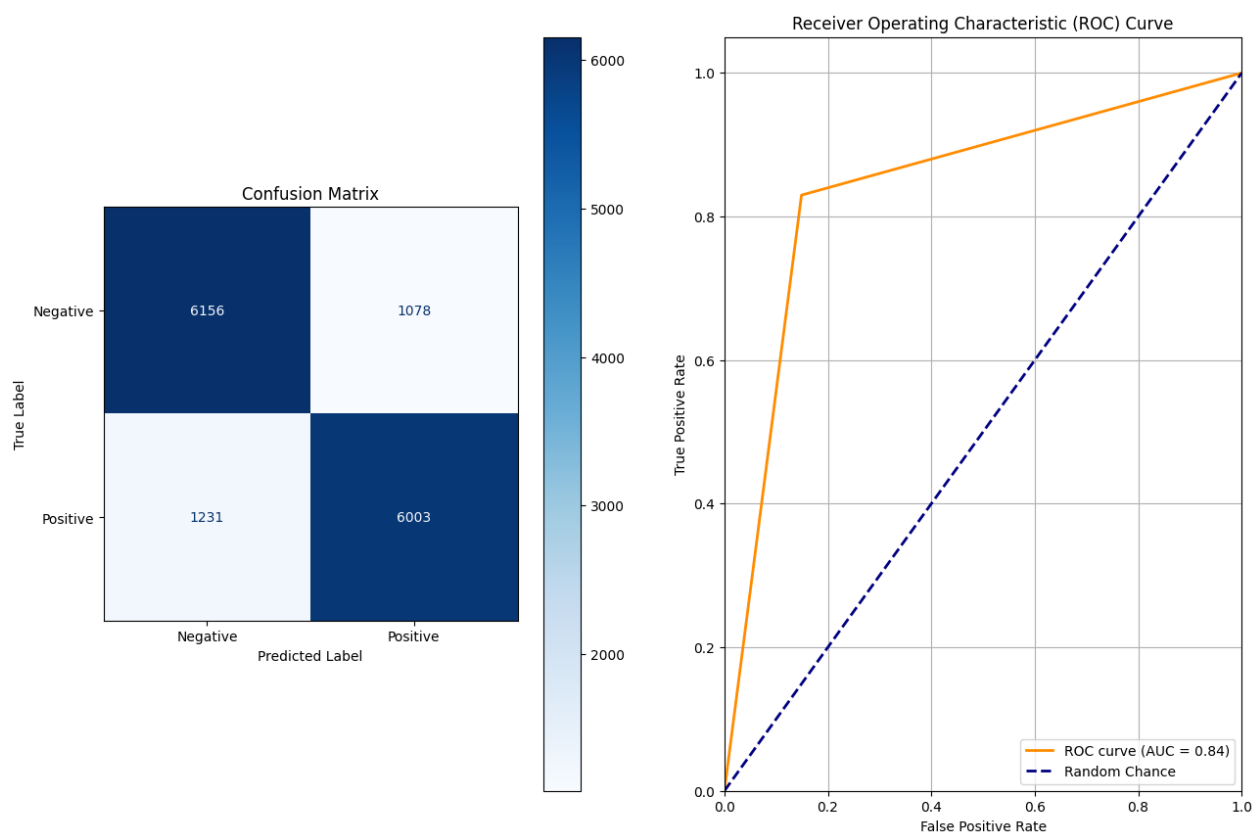


Image 5: CM and ROC Curve for Max Input Size Testing (2048)

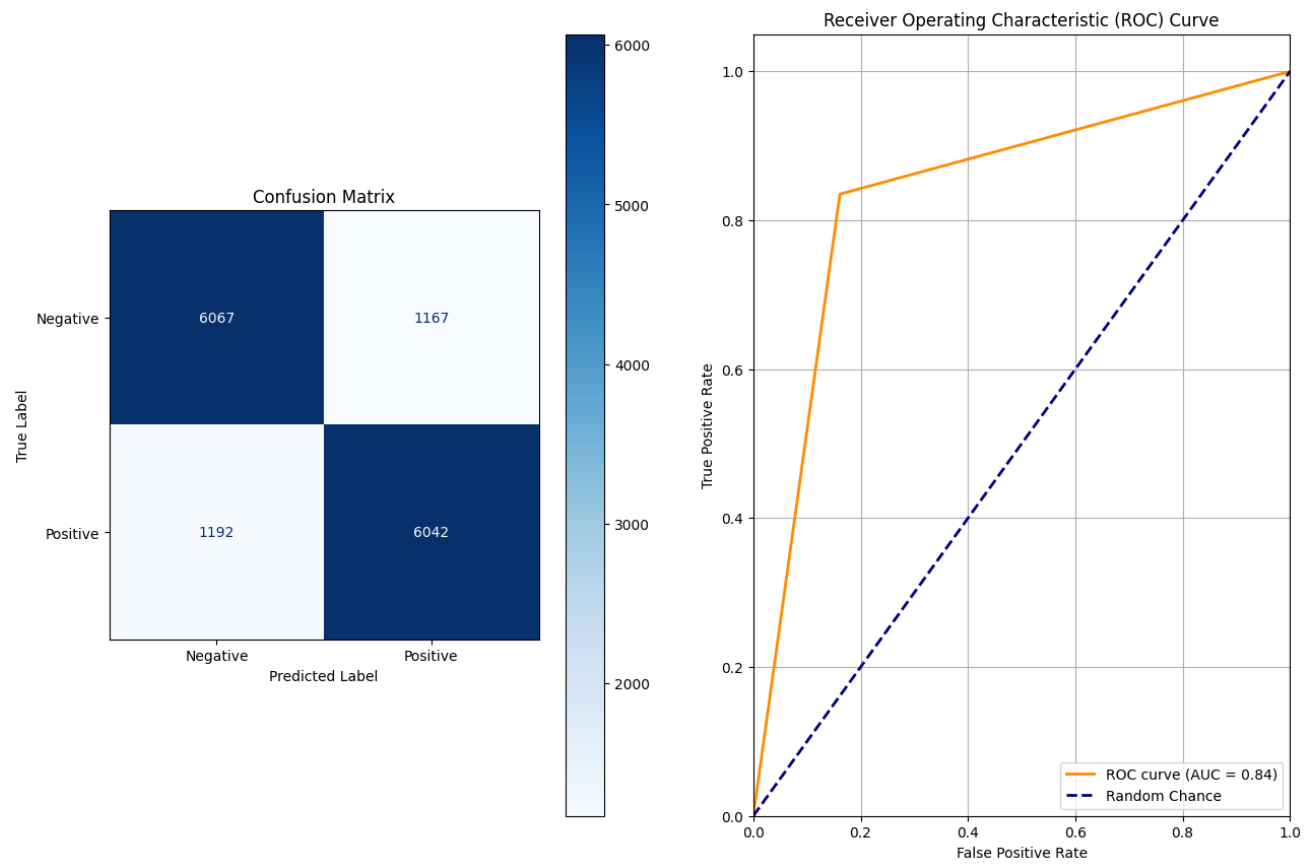


Image 6: CM and ROC Curve for Max Input Size Testing (1024)

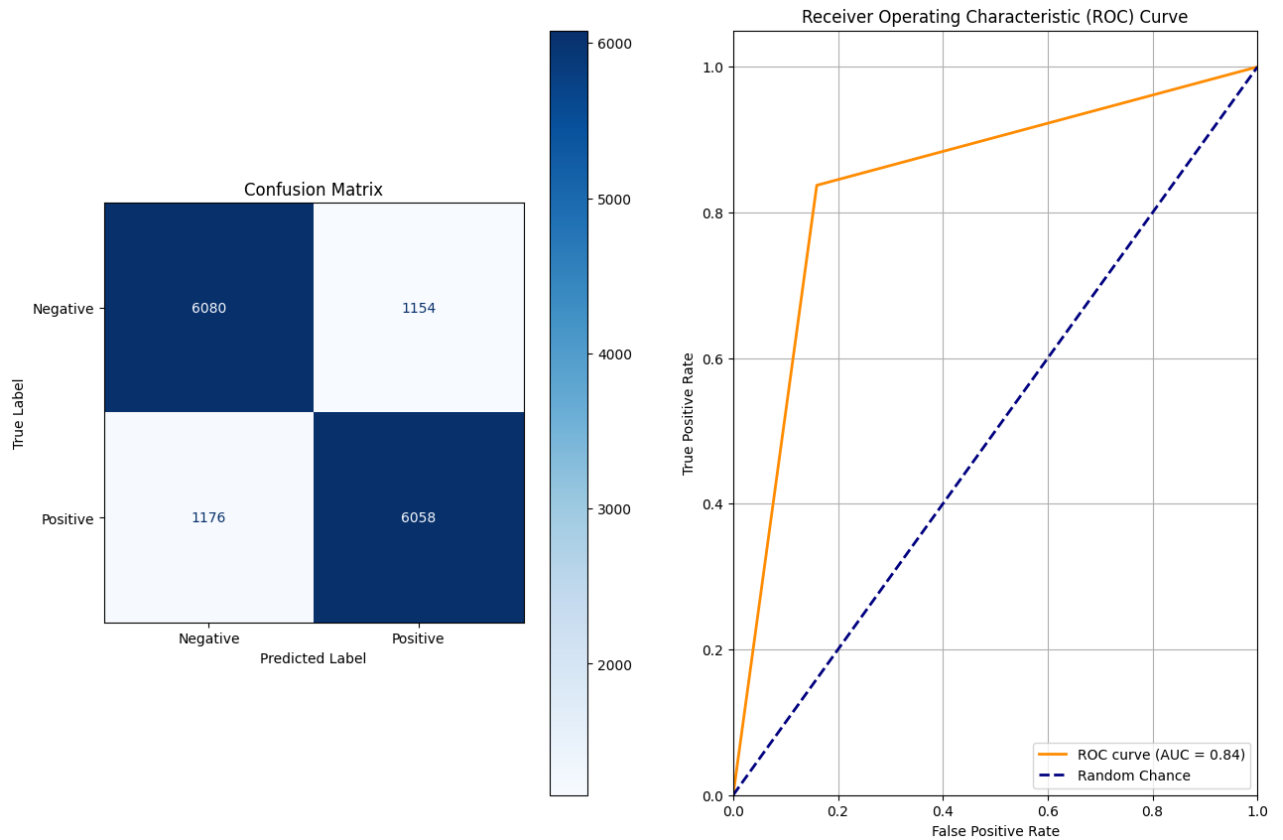


Image 7: CM and ROC Curve for Max Input Size Testing (512)

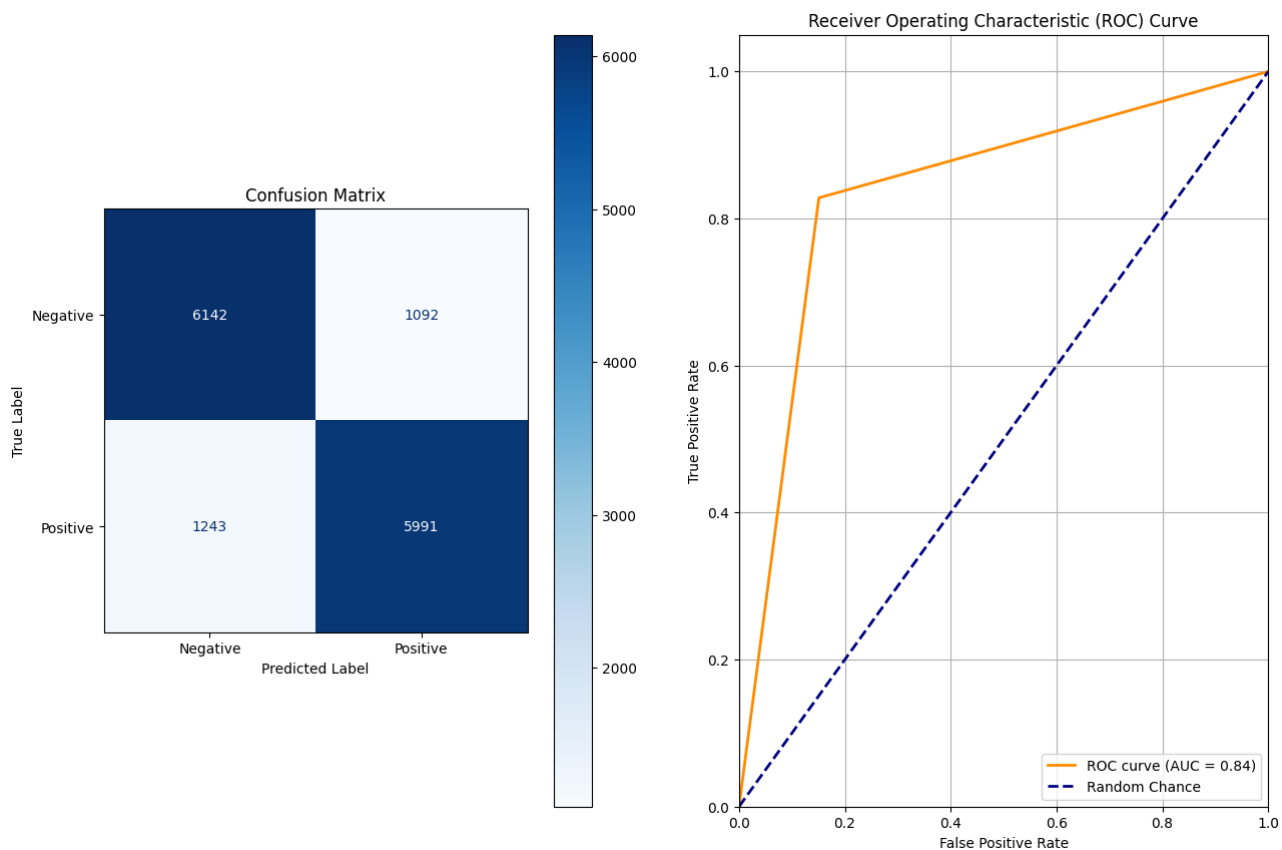


Image 8: CM and ROC Curve for Max Input Size Testing (256)

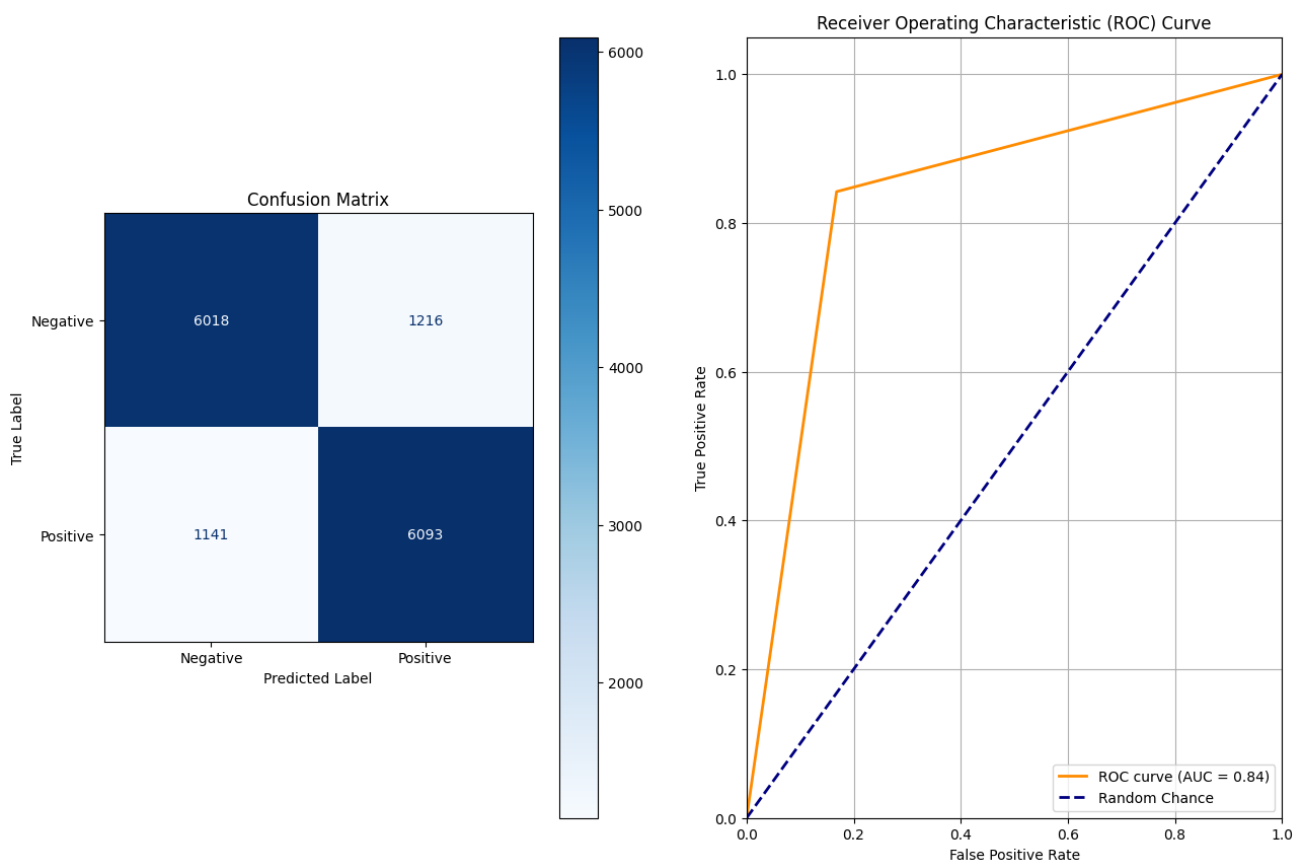


Image 8: CM and ROC Curve for Max Input Size Testing (128)

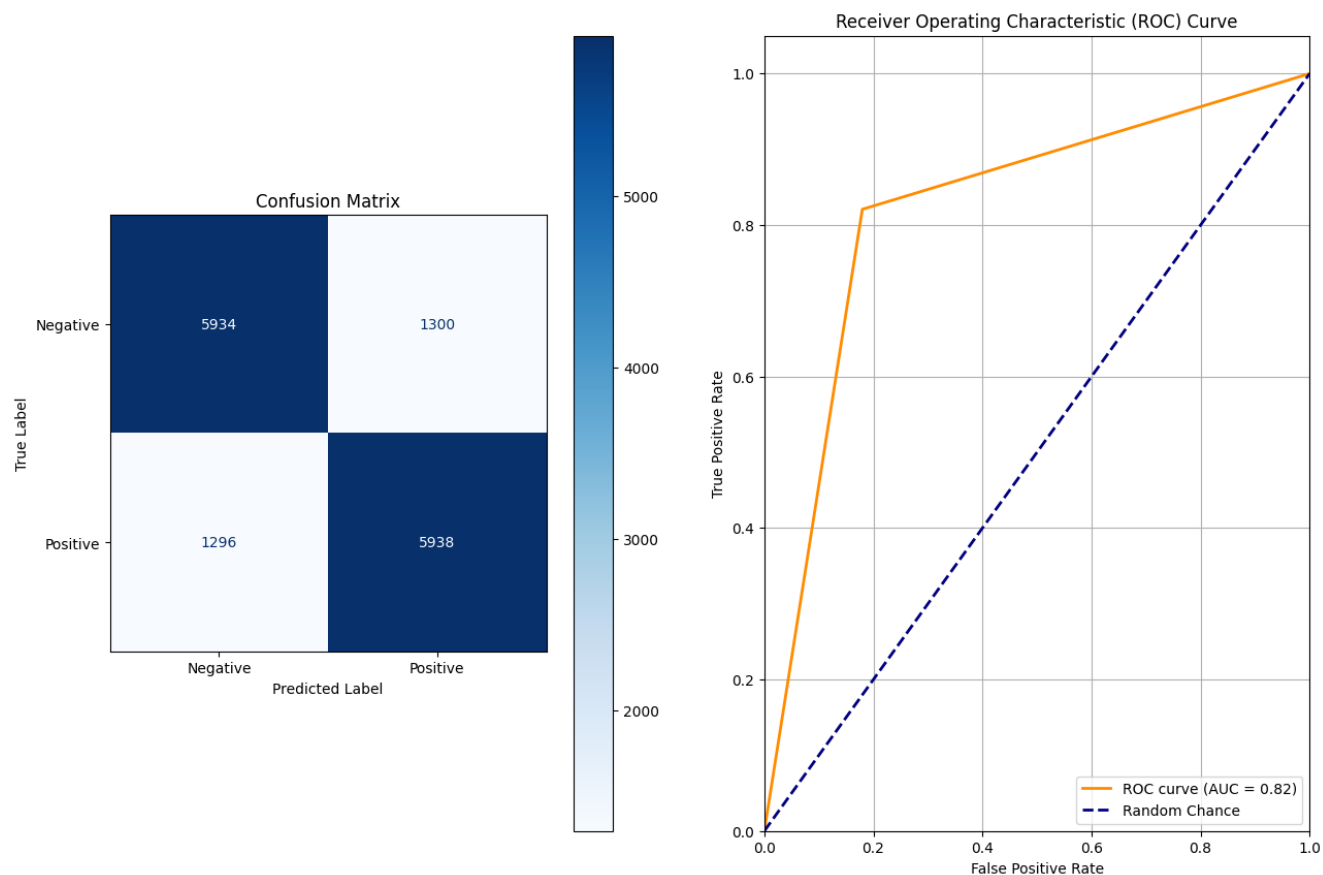


Image 9: CM and ROC Curve for Layer Freezing Testing (Layer 11)

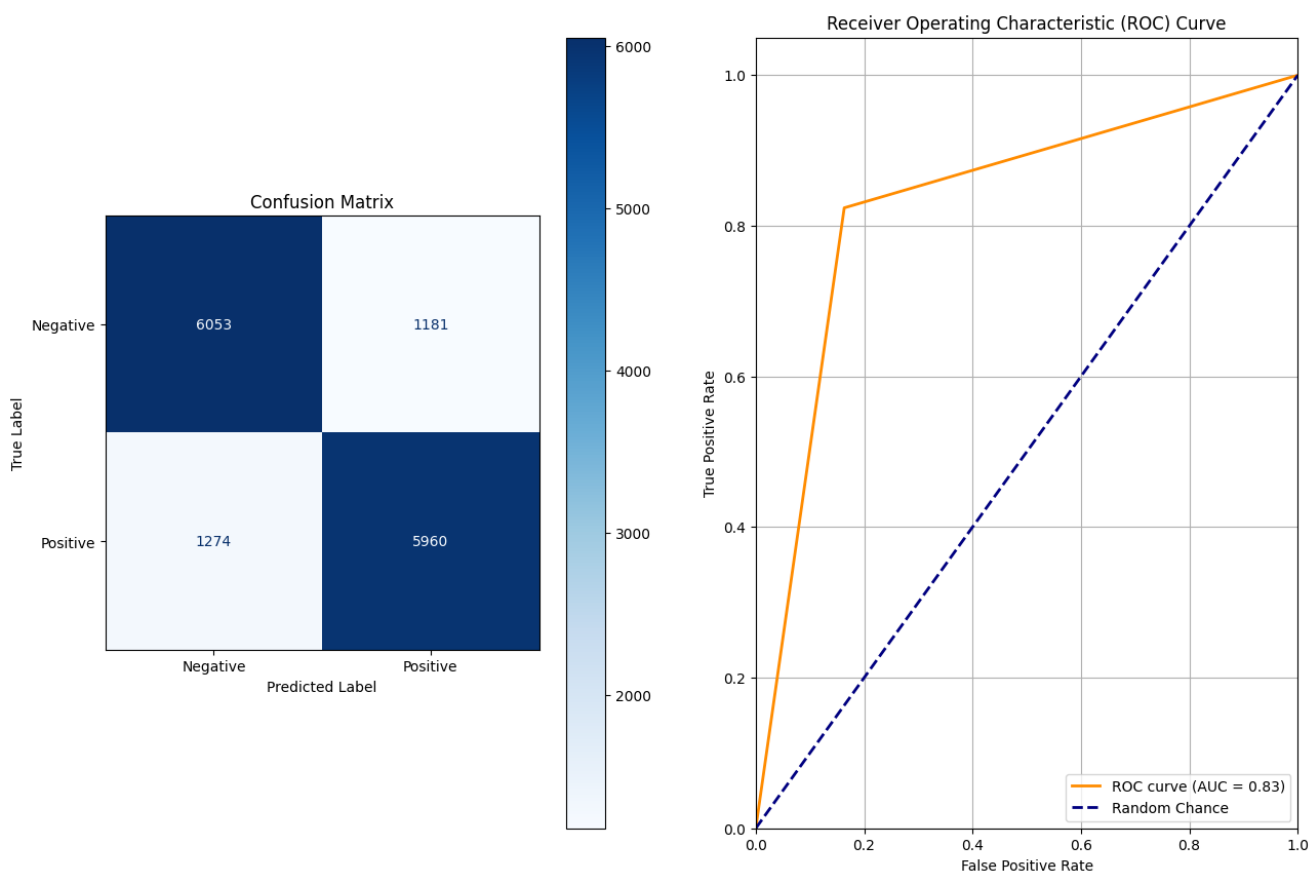


Image 10: CM and ROC Curve for Layer Freezing Testing (Layers 9, 10, 11)

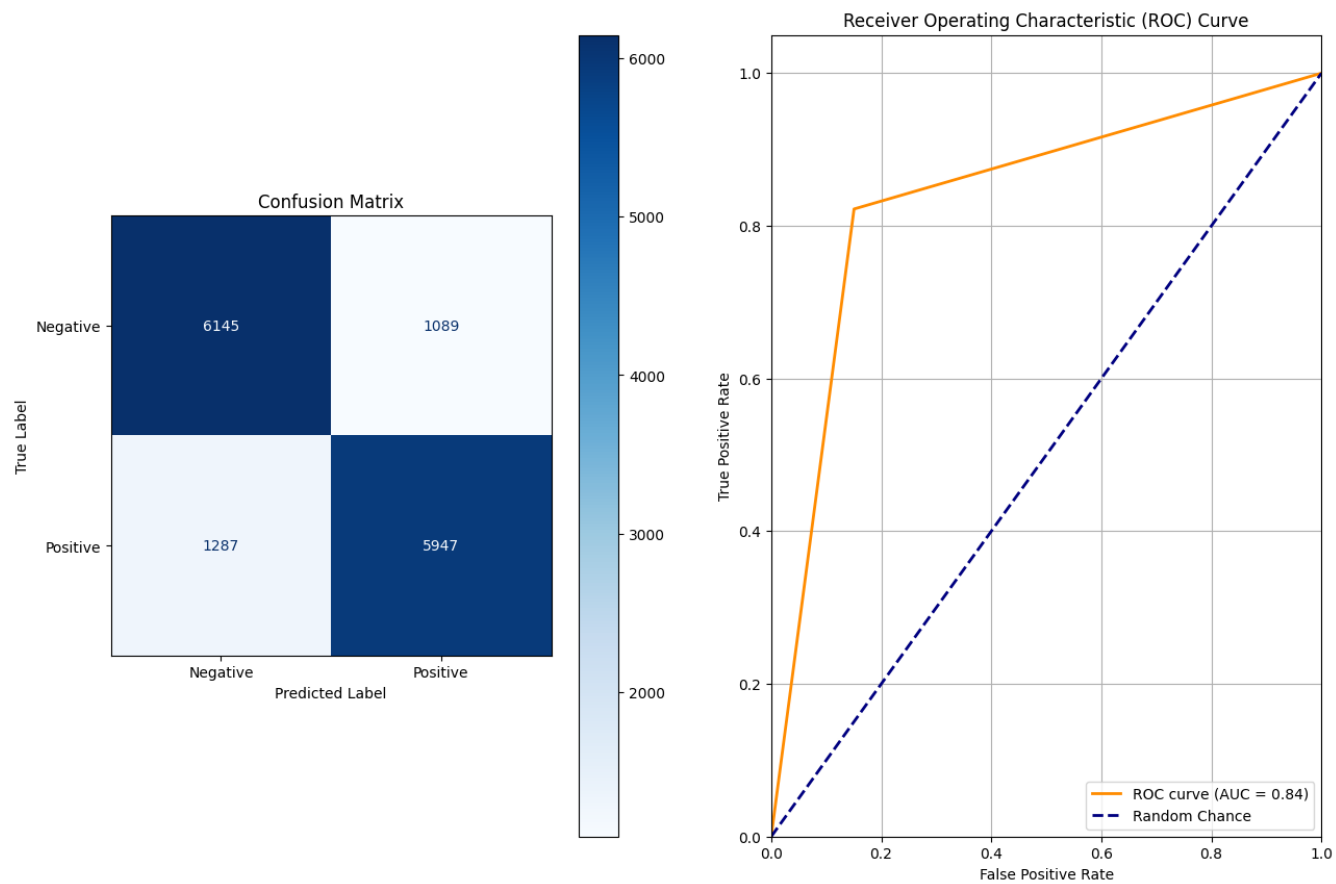


Image 11: CM and ROC Curve for Layer Freezing Testing (Layers 6, 7, 8, 9, 10, 11)